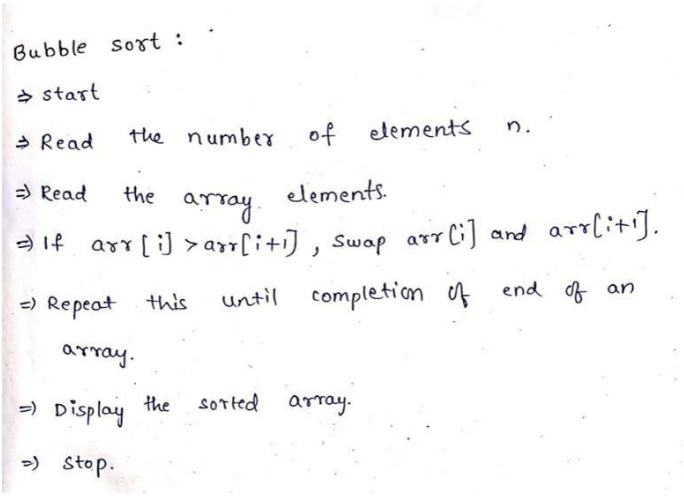
PRACTICAL-01

AIM: To implementation and time analysis of sorting algorithms.Bubble sort,Selection Sort,Insertion sort,Merge sorta and Quick sort.

1.BUBBLE SORT

ALGORITHM:



PROGRAM:

```
#include   <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
```

```
}

int main() {
int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    bubbleSort(arr, n);

    cout << "Sorted elements: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter array elements: 7
2
9
1
5
Sorted elements: 1 2 5 7 9
```

TIME COMPLEXITY:

```
for (int i=0 ; i<n-1 ; i++)..... outer loop
  {
    for (int j=0 ; j <n-1 ; j++)..... inner loop.
      {
        if (arr[j] > arr[j+1])..... 1
          { int temp = arr[j];----1
            arr[j] = arr[j+1];-----1
             arr[j+1] = temp ;-----1
          }
      }
  }
```

⇒ outer loop : Runs i=0 to n-1
so it executes n-1 times.

⇒ inner loop : pass (i) iterations

0	n-1
1	n-2
⋮	
n-2	1

$(n-1)+(n-2)+\ldots+2+1 = 1+2+3+\ldots+(n-1) = \frac{n(n-1)}{2}$

$= \frac{n^2-n}{2}$

$T(n) = n-1 + \frac{n^2-n}{2} + 1+1+1+1$

$= n + \frac{n^2-n}{2} + 3$

$= \frac{2n+n^2-n+6}{2} = \frac{n^2-n+6}{2}$ ignore all constant by asymptotic notation ($\because 6,2$)

$= n^2-n$

$T(n) = O(n^2)$ highest power.

worst case

condition: Array is in descending order

process : Every comparison leads to swap.

Comparisons: $(n-1)+(n-2)+\ldots+1 = \frac{n(n-1)}{2}$

$\frac{n(n-1)}{2} = \frac{n^2-n}{2} = n^2-n$

Time complexity $= O(n^2)$

Average case:

condition: Array element are in random order.

process : About half of the comparisons leads to swaps.

Comparisons: still $\frac{n(n-1)}{2}$ Time complexity : $O(n^2$

Best case

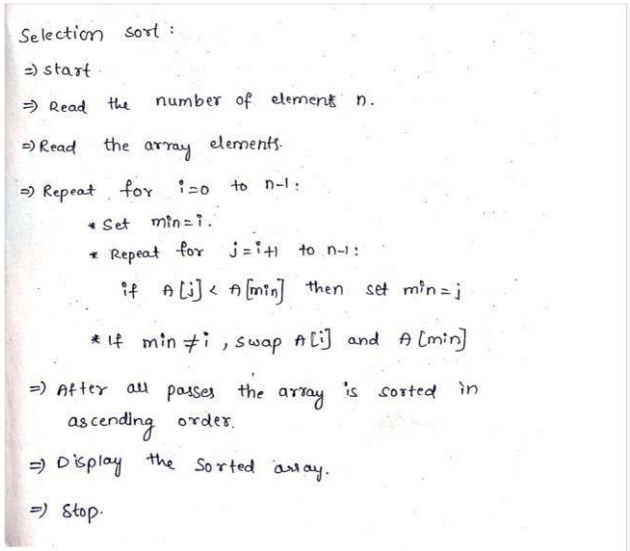
condition : Array is already sorted

process : comparisons occurs but no swaps.

Time complexity : $O(n)$

2.SELECTION SORT

ALGORITHM:


Selection sort :
⇒ start
⇒ Read the number of element n.
⇒ Read the array elements.
⇒ Repeat for i=0 to n-1:
* Set min=i.
* Repeat for j=i+1 to n-1:
if A[j] < A[min] then set min=j
* If min ≠ i, swap A[i] and A[min]
⇒ After all passes the array is sorted in ascending order.
⇒ Display the Sorted array.
⇒ Stop.

PROGRAM:

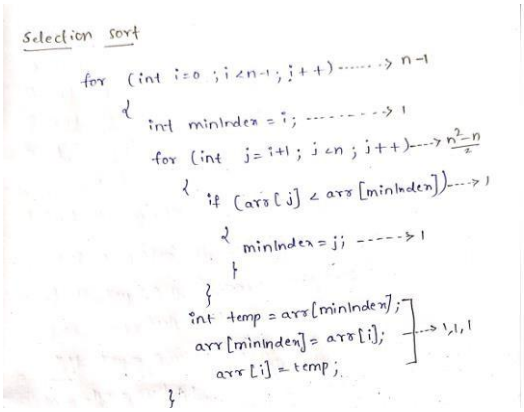
```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
int main() {
```

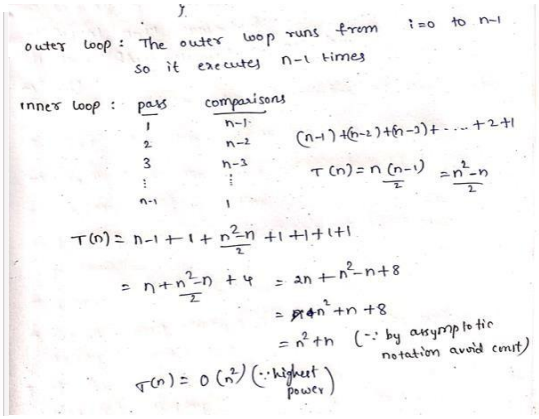
```
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "Original array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
        cout << endl;
    }
   selectionSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    //cout << endl;
    }
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 9
2
4
6
0
Original array: 9
2
4
6
0
Sorted array: 0 2 4 6 9
```

TIME COMPLEXITY:





worst case
⇒ Array is in descending order
⇒ For each position, the algorithm searches the entire unsorted part to find the minimum.
⇒ comparisons : (n-1) + (n-2) + ... + 1 = n(n-1)/2
⇒ swaps : At most n-1
⇒ Time complexity : O(n²)

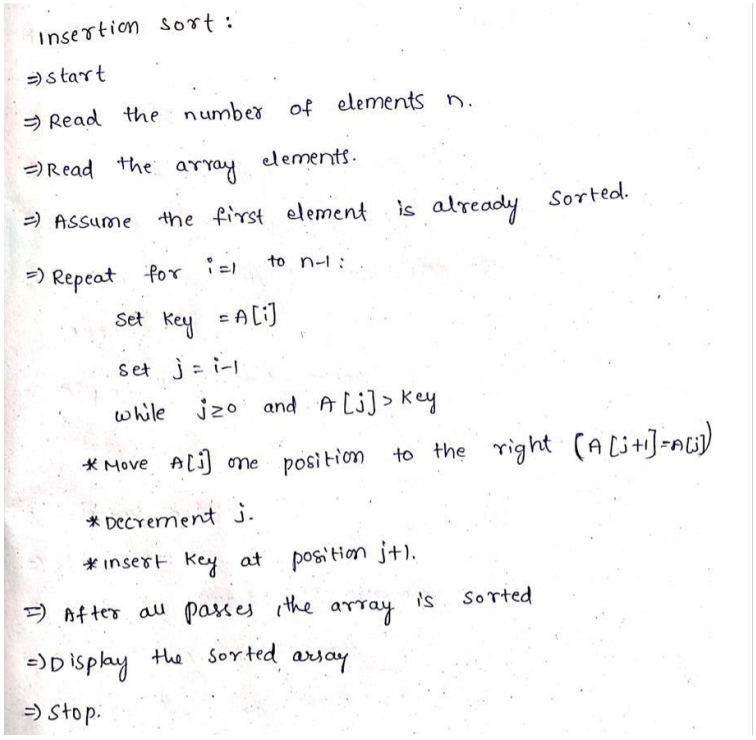
Average case
⇒ Array elements are in random order.
⇒ still scans the unsorted part fully for each position
⇒ comparisons : Same as worst case, n(n-1)/2
⇒ swaps : n-1
⇒ Time complexity : O(n²)

Best case
⇒ Array is already sorted.
⇒ Even though no swaps are needed, the algorithm still scans the unsorted part
⇒ comparisons : same as worst case, n(n-1)/2
⇒ swaps : 0
⇒ Time complexity : O(n²)

bat

3.INSERTION SORT

ALGORITHM:


Insertion sort :
⇒ start
⇒ Read the number of elements n.
⇒ Read the array elements.
⇒ Assume the first element is already sorted.
⇒ Repeat for i=1 to n-1 :
Set key = A[i]
set j = i-1
while j≥0 and A[j] > key
* Move A[i] one position to the right (A[j+1]=A[j])
* Decrement j.
* insert key at position j+1.
⇒ After all passes the array is sorted
⇒ Display the sorted array
⇒ Stop.

PROGRAM:

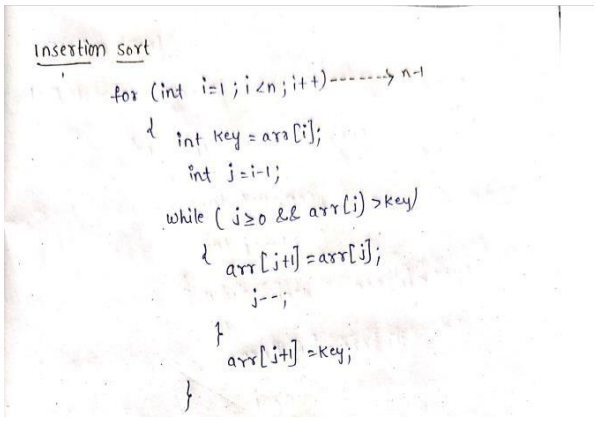
```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
```

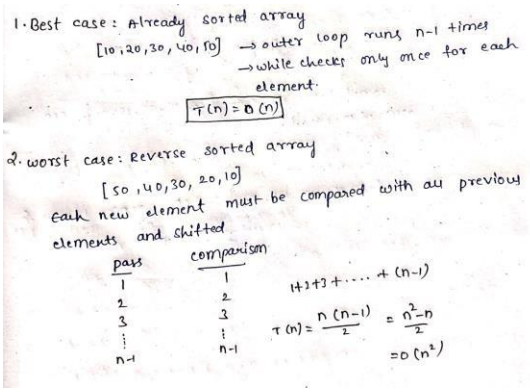
```
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Original array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
        cout << endl;
    }
    insertionSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
        //cout << endl;
    }
    return 0;
}
```

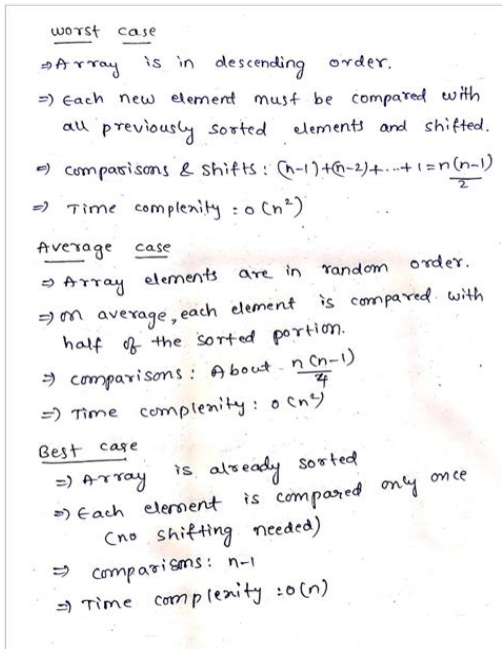
OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 7
3
8
2
0
Original array: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

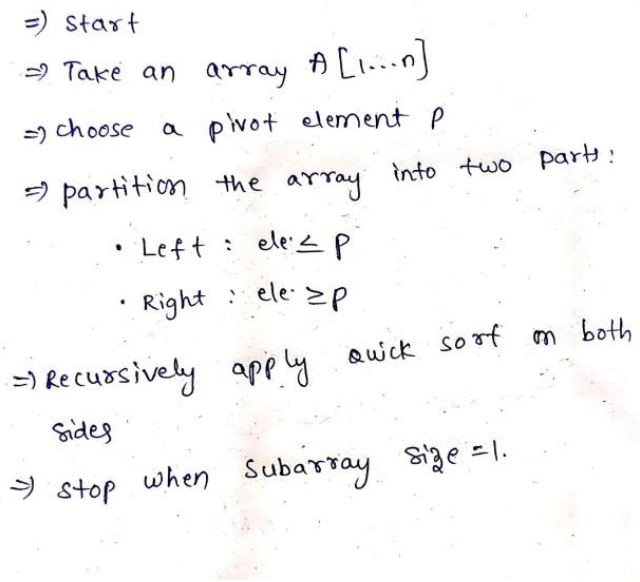






4.QUICK SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;
int partition(int arr[], int lb, int ub) { int
    pivot = arr[lb];
    int start = lb; int
    end = ub;

    while (start < end) {
        while (arr[start] <= pivot ) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
```

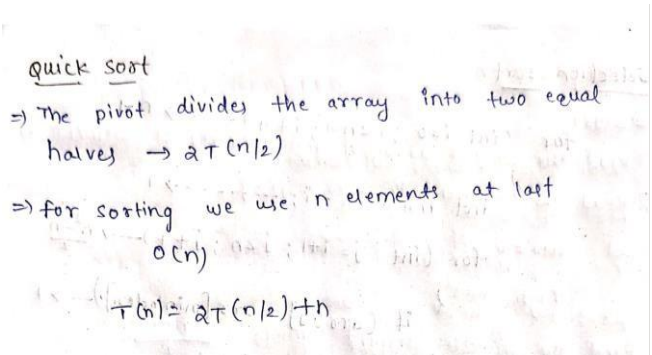
```
        }

        if (start < end) {
            swap(arr[start], arr[end]);
        }
    }
    swap(arr[lb], arr[end]);
    return end;
}

void quickSort(int arr[], int lb, int ub) { if
    (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1); quickSort(arr,
        loc + 1, ub);
    }
}
int main() {
    int n;
    cout << "Enter size of array: "; cin
    >> n;

    int arr[n];
    cout << "Enter " << n << " elements: "; for
    (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
```

```
    quickSort(arr, 0, n - 1);

   cout << "Sorted array: "; for
     (int i = 0; i < n; i++) { cout
            << arr[i] << " ";
    }
    return 0;
}
```

OUTPUT:

```
Enter size of array: 5
Enter 5 elements: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:



$T(n) = 2T(n/2) + n \rightarrow (1)$

$n \rightarrow n/2$ put in (1)

$T(n/2) = 2T(n/4) + n/2$

$T(n) = 2T(n/4) + n/2 \rightarrow (2)$

~~put~~ sub (2) in (1)

$T(n) = 2(2T(n/4) + n/2) + n$

$= 2^2 T(n/4) + \frac{2n}{2} + n$

$= 2^2 T(n/4) + 2n \rightarrow (3)$

$n \rightarrow n/4$ put in (3)

$T(n/4) = 2^2 T(n/2^4) + 2(n/4) \rightarrow (4)$

(4) in (3)

$T(n) = 2^2\left(2^2 T(n/2^4) + \frac{2n}{4}\right) + 2n$

$= 2^4 T(n/2^4) + \frac{2^3 n}{4} + 2n$

$= 2^4 T(n/2^4) + 4n$

$T(n) = 2^k T(n/2^k) + kn$

$T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1$

$n = 2^k$

Apply log both sides

$\log n = \log 2^k$

$k = \log n$

$T(n) = 2^{\log n} T(n/2^{\log n}) + \log n(n)$

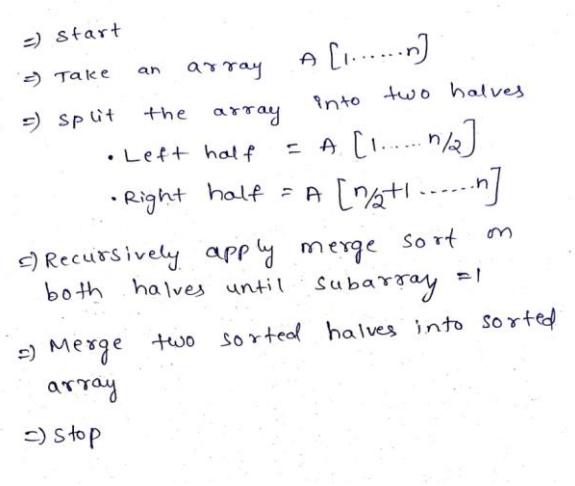
$= n^{\log 2} T\left(\frac{n}{n^{\log 2}}\right) + \log n(n)$

$= n \log n + n$

$T(n) = O(n \log n)$

5.MERGE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;


void merge(int arr[], int left, int mid, int right) { int
    n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
     while (i < n1 && j < n2) {
       if (L[i] <= R[j]) arr[k++] = L[i++]; else
       arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
int main() { int
    n;
    cout << "Enter number of elements: "; cin
    >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements: 7
9
0
2
5
Sorted array: 0 2 5 7 9
```

TIME COMPLEXITY:

Merge sort

⇒ The array is divided into two halves repeatedly

Number of divisions : $\log_2 n$ ($2T(n/2)$

⇒ merge : each level merges all n elements

$O(n)$

Total time $T(n) = 2T(n/2) + n$

$T(n) = 2T(n/2) + n \rightarrow (1)$

$n \rightarrow n/2$ put in (1)

$T(n/2) = 2T(n/4) + n/2$

$T(n) = 2T(n/4) + n/2 \rightarrow (2)$

put sub (2) in (1)

$T(n) = 2(2T(n/4) + n/2) + n$

$= 2^2 T(n/4) + 2\frac{n}{2} + n$

$= 2^2 T(n/4) + 2n \rightarrow (3)$

$n \rightarrow n/4$ put in (3)

$T(n/4) = 2^2 T(n/2^4) + 2(n/4) \rightarrow (4)$

(4) in (3)

$T(n) = 2^2\left[2^2 T(n/2^4) + \frac{2n}{4}\right] + 2n$

$= 2^4 T(n/2^4) + \frac{2^3 n}{4} + 2n$

$= 2^4 T(n/2^4) + 4n$

$T(n) = 2^k T(n/2^k) + kn$

$T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1$

$n = 2^k$

Apply log both sides

$\log n = \log 2^k$

$k = \log n$

$T(n) = 2^{\log n} T(n/2^{\log n}) + \log n(n)$

$= n^{\log 2} T\left(\frac{n}{n^{\log 2}}\right) + \log n(n)$

$= n \log n + n$

$T(n) = O(n \log n)$

CONCLUSION:

Bubble Sort and Selection Sort are simple but inefficient with $O(n^2)$in all major cases. Insertion Sort improves to $O(n)$in the best case when the array is already sorted. Quick Sort is very efficient on average with $O(n \log n)$, though it can degrade to $O(n^2)$. Merge Sort is consistently $O(n \log n)$in all cases but requires extra space.